# AMQP

## ADVANCED MESSAGE QUEUING PROTOCOL



¿API?

¿Protocolos?

¿Mensajes?

Presentado por:
Carlos Andres Celis H. Xiomara Salome Arias A. Cesar Augusto Pulido C.

Asignatura: Web Backend

Fecha

26 Octubre, 2026

# CONTEXTO




INICIOS



En 2003, **John O'Hara**, ingeniero de JPMorgan Chase, impulsó la creación de AMQP a partir de su experiencia desarrollando sistemas de procesamiento bancario. Según relata en su artículo *Toward a Commodity Enterprise Middleware*, estaba cansado de que los mismos **problemas de comunicación entre sistemas** se repitieran y de depender de soluciones de middleware propietarias y costosas.

Entre 1996 y 2003, esperó que surgiera un estándar abierto que permitiera resolver estos problemas, pero, ante la falta de una solución, decidió impulsar AMQP. Su objetivo era establecer un protocolo abierto e interoperable que facilitara la comunicación entre aplicaciones, redujera la dependencia de proveedores y permitiera a las organizaciones implementar soluciones de mensajería más eficientes y económicas.

# CONTEXTO...










**INICIOS**

Inicialmente, se utilizó junto con iMatrix Corporation para desarrollar la documentación del protocolo y sistemas de intermediación de mensajes en C.

**EVOLUCIÓN**

En 2005, JPMorgan Chase impulsó su adopción y promovió la creación de un grupo de trabajo con empresas como Cisco, Red Hat, IONA Technologies y TWIST. Con el tiempo, este grupo creció hasta reunir 23 compañías, entre ellas Barclays, Bank of America y Credit Suisse.

**APROBACIÓN**

En octubre de 2011 se publicó AMQP 1.0. Ese mismo año se presentó ante OASIS, organización encargada de supervisar su estandarización. Tras varias revisiones durante 2012, AMQP fue aprobado como estándar OASIS en octubre de ese año. Finalmente, en 2014, el protocolo fue reconocido como estándar internacional por la IEC y la ISO.

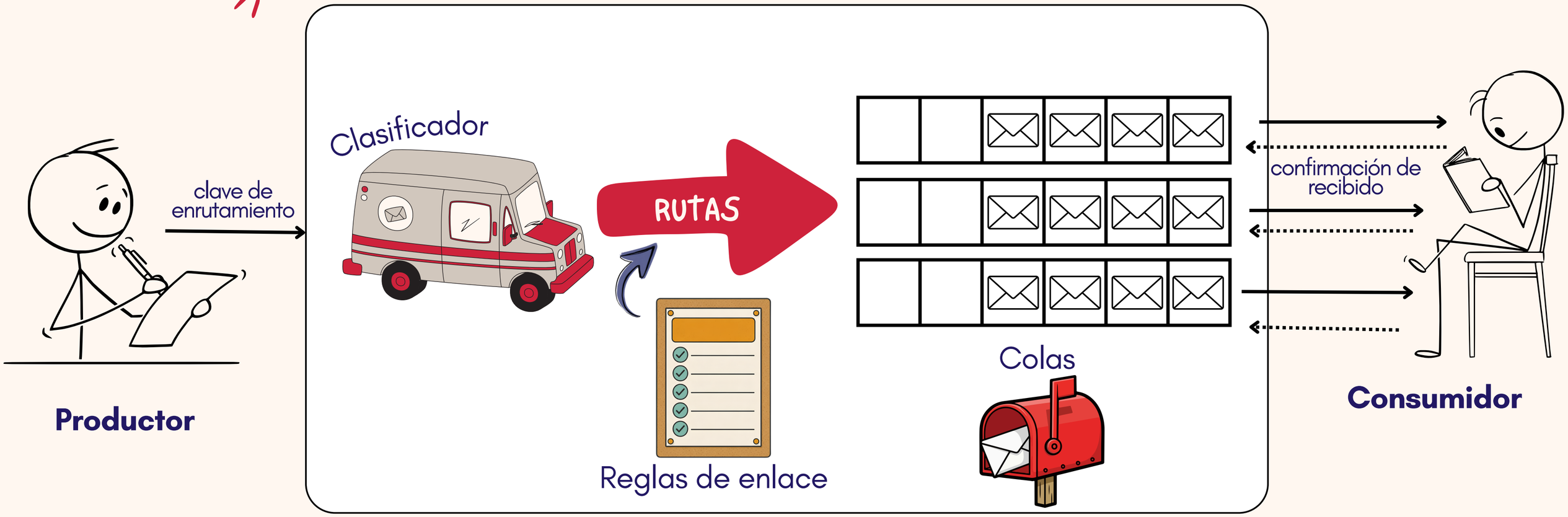








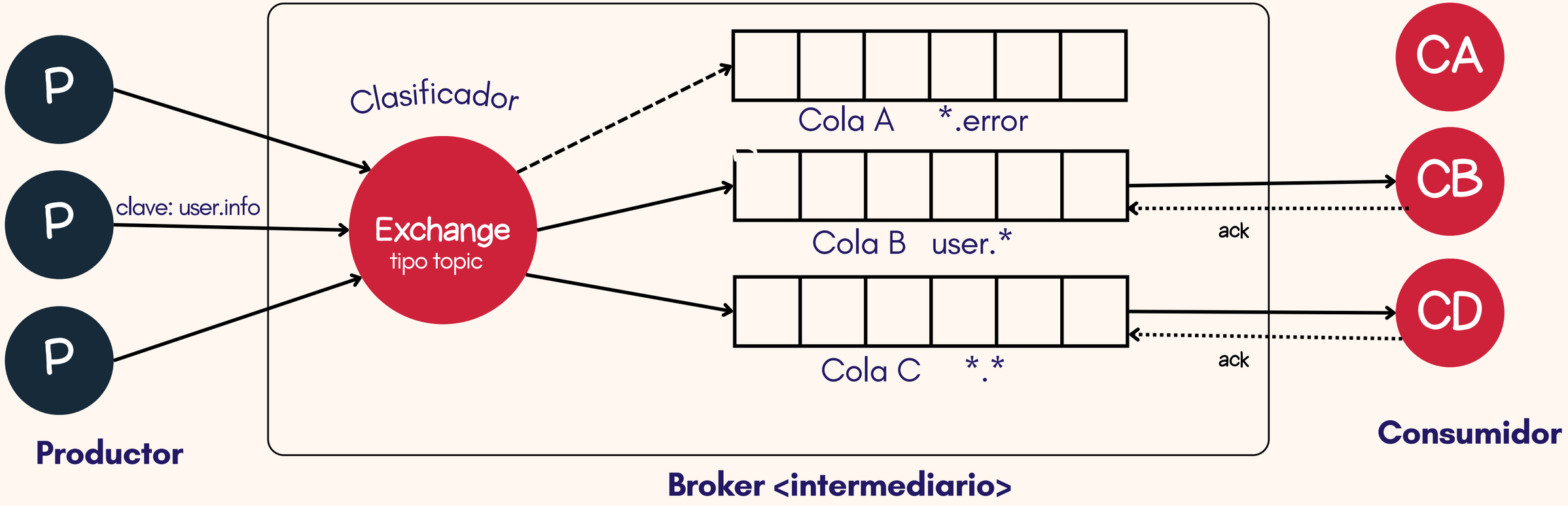
# ¿CÓMO FUNCIONA AMQP?




Como en el correo: el productor escribe la carta con una dirección (clave de enrutamiento), el exchange (clasificador) la dirige según las reglas del binding y la deja en una cola (buzón). El consumidor la recoge y confirma con un ack.

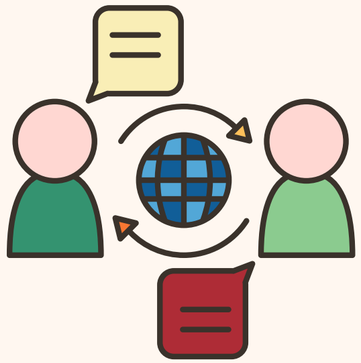
# ¿CÓMO FUNCIONA AMQP?

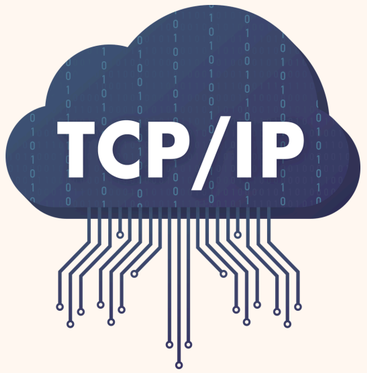
| Tipo Exchange | Cómo reparte | Analogía postal |
|---|---|---|
| **Direct (directo)** | Solo a la cola cuya regla coincide exactamente con la clave. | Carta a una dirección exacta. |
| **Fanout (difusión)** | A todas las colas, ignora la clave. | Folleto que llega a todos los buzones. |
| **Topic (por temas)** | A las colas cuyo patrón coincide con la clave (usa * y #). | Suscripción por tema: "solo noticias de deportes". |
| **Headers (por cabeceras)** | Según las cabeceras del mensaje, no por la clave. | Clasificar por lo que dice la etiqueta del paquete, no por la dirección. |




Todo viaja sobre TCP (puerto 5672) en formato binario, en tramas

# ¿CÓMO FUNCIONA?






## Modelo de comunicación

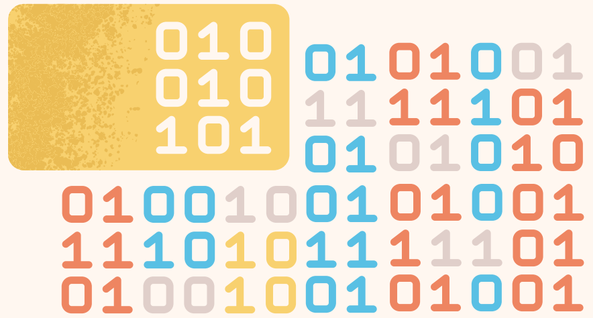
Publicación/suscripción asíncrona con colas.
El productor envía y no espera respuesta.

## Protocolo de transporte

TCP, puerto 5672 (5671 si usa cifrado TLS).

## Formato de los datos

Binario, en tramas (frames). El contenido del mensaje puede ser JSON, texto, etc.



# VOCABULARIO CLAVE



01 **publisher → productor**

Quien crea el mensaje y lo envía. Es quien escribe la carta.

**Broker → intermediario** 02

El servidor que recibe, clasifica y guarda los mensajes hasta entregarlos. Es el servicio de correos.

03 **Exchange → clasificador**

Recibe los mensajes y decide a qué cola van. No los guarda, solo los reparte.

**Queue → cola** 04

Lugar donde el mensaje espera hasta que un consumidor lo recoge. Es el buzón.

**Binding → regla de enlace** 05

Regla que conecta un exchange con una cola y define qué mensajes le corresponden.

06 **Routing key → clave de enrutamiento**

Etiqueta del mensaje, como la dirección en el sobre. El exchange la compara con los bindings.

# VENTAJAS

## Confiabilidad

Transmisión sin pérdidas incluso durante periodos de indisponibilidad temporal

## Flexibilidad de Enrutamiento

Enrutamiento potente gracias a los iclasificadores, las colas y las reglas de enlaces

## Seguridad

TLS integrado, autenticación y controles de acceso detallados

## Transacción Segura

Compatibilidad con transacciones con capacidad de reversión

## Alcance de Aplicación

Especialmente adecuado para aplicaciones críticas que requieran estabilidad, seguridad y auditabilidad

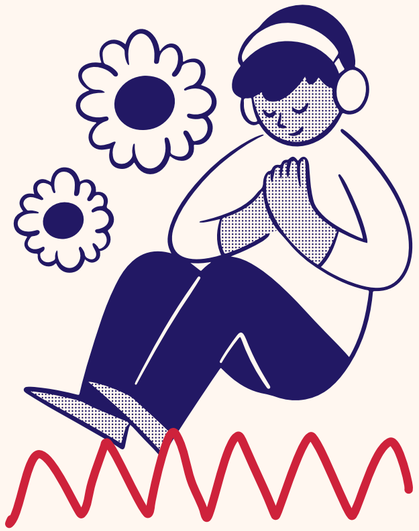
## Arquitectura del Sistema

Enrutamiento de mensajes gestionado de forma centralizada




¿ES HORA DE COMENZAR A USARLO?





En una tienda en línea, el servicio de facturación se cae 10 minutos por un despliegue. Los pedidos nuevos quedan en una cola durable y, cuando el servicio vuelve, los procesa todos. Ningún pedido se pierde.

**Confiabilidad**

Un pedido nuevo se publica una sola vez en un exchange. Gracias a los bindings, llega a la cola de facturación, a la de inventario y a la de notificaciones por correo, sin que el productor sepa quiénes consumen.

**Enrutamiento**

El servicio de pagos puede publicar en la cola pagos, pero el de reportes solo puede leer de reportes. Cada uno tiene su usuario y permisos, y la conexión va cifrada con TLS.

**Seguridad**

# EN LA VIDA



NO!! HOMBRE TULIO! ESTE PROTOCOLO CHUZCO ES

Ejemplos elaborados con apoyo de Claude (Anthropic, 2026).

# DESVENTAJAS

## Confiabilidad

Requiere una configuración y un ajuste complejos del sistema

## Flexibilidad de Enrutamiento

Una implementación más compleja en comparación con los protocolos más sencillos

## Seguridad

Mayor consumo de recursos (uso de la CPU y la memoria)

## Transacción Segura

Mayor esfuerzo en materia de desarrollo y operaciones

## Alcance de Aplicación

Menos adecuado para sistemas sencillos o con recursos limitados

## Arquitectura del Sistema

Una estructura de broker central puede constituir un punto único de fallo, por lo que se requiere redundancia adicional.





Para que el sistema sea confiable hay que definir colas durables, mensajes persistentes, confirmaciones (acks), reintentos y dead-letter queues. .

**Confiabilidad**

Si se acumulan millones de mensajes en las colas, el broker consume mucha memoria y puede bloquear a los productores. No es ideal para correr en un dispositivo IoT pequeño o una Raspberry Pi.

**Enrutamiento**

Si el único nodo de RabbitMQ se cae, ningún servicio puede enviar ni recibir mensajes y todo el sistema se paraliza. Por eso se necesita redundancia: un clúster de varios nodos con colas replicadas (quorum queues), lo que añade costo y complejidad operativa.

**Seguridad**

# EN LA VIDA



NO!! HOMBRE TULIO! EL PROTOCOLO SE NOS CAYO

Ejemplos elaborados con apoyo de Claude (Anthropic, 2026).

# ¿CUÁNDO USARLO?

"El mejor protocolo lo va determinar las necesidades del proyecto."




1. Comunicación entre servicios
2. Recursos extensos
3. Fiabilidad alta en los mensajes
4. Enrutados más complejos

## ÁREAS TIPICAS DE APLICACIÓN

- **Arquitecturas orientadas a servicios.**
- **Sistemas distribuidos.**
- **Infraestructuras críticas para la seguridad.**

# CASOS DE USO REAL

El servicio en la nube Azure Service Bus usa AMQP 1.0 como su medio principal de comunicación.

Usaron Celery como framework de tareas y RabbitMQ, un broker AMQP, para asegurar la entrega de mensajes a los workers

Streaming de eventos de altísimo volumen con repetición de mensajes. Una plataforma que registra los clics de millones de usuarios y necesita releer el historial.



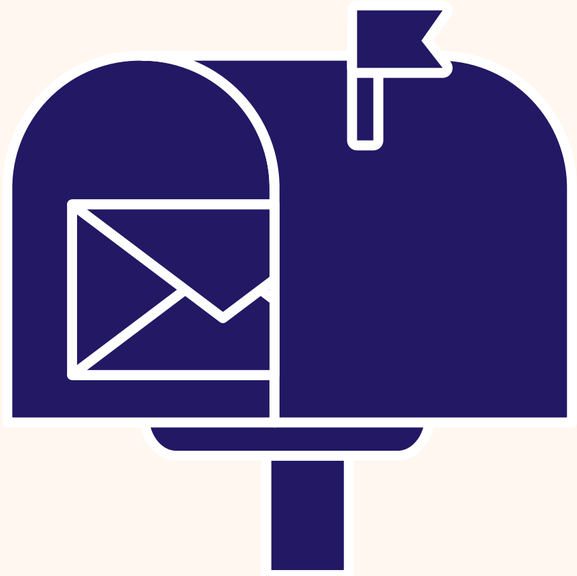
...

Ejemplos elaborados con apoyo de Claude (Anthropic, 2026).

# REST O AMQP ¿=?



Vs



# ¿CÓMO SE COMUNICAN?




REST




AMQP

# COMPARACIÓN CON REST

| REST | | AMQP |
|---|---|---|
| Petición y respuesta (síncrono) | MODELO | Mensajería asíncrona con broker |
| HTTP | TRANSPORTE | Protocolo propio sobre TCP. |
| Normalmente JSON, legible | DATOS | Protocolo binario, eficiente. |
| Alto: el cliente espera a que el servidor responda | ACOPLAMIENTO | Bajo: emisor y receptor no tienen que estar activos a la vez |
| La petición falla. | RECEPTOR CAE | El mensaje espera en la cola hasta que el receptor vuelva |

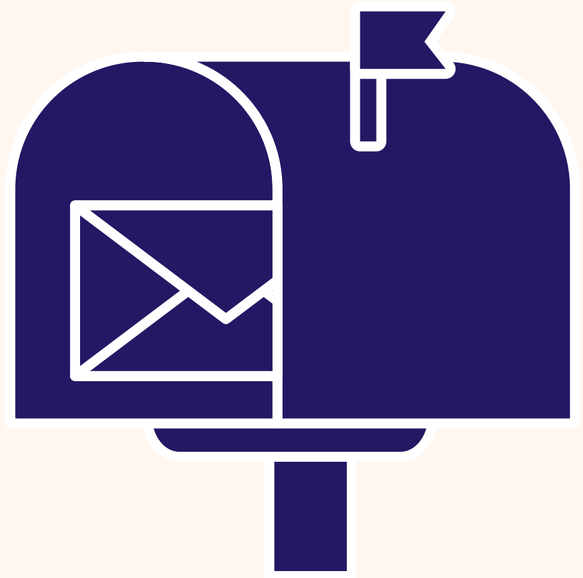
# ¿CUÁNDO GANA REST?



- NECESITAS LA RESPUESTA YA
- API PÚBLICA O PARA EL NAVEGADOR
- SISTEMA SIMPLE

# ¿CUÁNDO GANA AMQP?

- EL TRABAJO PUEDE ESPERAR
- UN EVENTO, VARIOS SERVICIOS
- HAY QUE RESISTIR CAÍDAS



# REFERENCIAS


1. Anthropic. (2026). Claude (Claude Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. Consistent practice. Conversación del 24 de octubre de 2026.
2. A1 Digital. (2025, 20 de junio). AMQP (Advanced Message Queuing Protocol) para la gestión de mensajes. https://www.a1.digital/es/knowledge-hub/amqp-advanced-message-queuing-protocol-para-la-gestion-de-mensajes/
3. Wallarm. (s. f.). What is AMQP? https://www.wallarm.com/what/what-is-amqp
4. CloudAMQP. (s. f.). What is AMQP and why is it used in RabbitMQ? https://www.cloudamqp.com/blog/what-is-amqp-and-why-is-it-used-in-rabbitmq/
5. O'Hara, J. (2007). Toward a commodity enterprise middleware. ACM Queue, 5(5), 48-55. https://doi.org/10.1145/1255421.1255424
6. Juan Nicolás. (2023, 1 de marzo). Qué son y cuándo usar los protocolos AMQP, MQTT y STOMP. Juan Nicolás. https://www.juannicolas.eu/que-son-y-cuando-usar-los-protocolos-amqp-mqtt-y-stomp/

# REFERENCIAS


7 AsyncAPI Initiative. (s. f.-a). AMQP 0-9-1 bindings. Anthropic. (2026). Claude (Claude Sonnet 5.5) [Modelo de lenguaje de gran tamaño]. Consistent practice. Conversación del 24 de octubre de 2026.

8 AsyncAPI Initiative. (s. f.-b). Coming from OpenAPI. https://www.asyncapi.com/docs/tutorials/getting-started/coming-from-openapi

9 CloudAMQP. (s. f.). Message queues vs HTTPS in microservices, part 1: A more decoupled architecture. Wallarm. (s. f.). What is AMQP? https://www.wallarm.com/what/what-is-amqp

10 DZone. (s. f.). REST vs. messaging for microservices: Choosing the right communication style for your microservices. https://dzone.com/articles/rest-vs-messaging-for-microservices



# CONCLUSIÓN

¿PREGUNTAS?